



中國人民大學
RENMIN UNIVERSITY OF CHINA

基于 Jittor 的时序图候选排序

赛道一 B 榜结果复现技术报告

团队	sader
作者	宋文韬 邓富骏 颜韬
学校	中国人民大学
竞赛成绩	赛道一 B 榜第 2 名 · 1.5240999401892983

摘要

本报告给出赛道一 B 榜记录结果的代码说明、模型训练协议、锁定重建过程和复验结果。B 榜延续 A 榜的 Jittor 时序图排序框架：历史交互形成时间统计和实体表示，学习成员给出候选分数，再在候选集合内部完成归一化、残差融合和稳定排序。针对 B 榜更大的实体词表，系统采用排序索引与流式批量前向，在保持模型训练、候选内融合和规范封装共同接口的同时控制显存与中间状态规模。

报告分别说明 Dataset3 的目标频次结构残差、Dataset4 的 MF32 隐式偏好模型、候选内 qnorm/tanh 校准、稳定序列化与工件校验。锁定复验路径和原始训练路径在调用语义上分离，并由清单、输入摘要、模型摘要、成员 CRC 和结果 SHA-256 形成可追踪证据链。

关键词 时序图；候选排序；Jittor；隐式反馈；候选内残差；可复现性

团队与成绩信息

字段	内容
团队名称	sader
赛道与 B 榜排名	赛道一；第 2 名
B 榜最优总分	1.5240999401892983
A 榜参考成绩	赛道一第 7 名；1.521072794155721
联系人	宋文韬
联系方式	微信 laoshan_song；电话 18984157192

目录

摘要	2
团队与成绩信息	2
目录	3
1 项目概述	5
1.1 任务定义与数据边界	5
1.2 双数据集建模思路	5
1.3 方法创新点与设计取舍	5
2 模型与算法	6
2.1 Dataset3: 目标频次结构残差	6
2.2 Dataset4: 稀疏交互表示与隐式偏好	6
2.3 负采样与训练目标	7
2.4 参数规模与流式推理	7
3 候选内校准、融合与输出	7
3.1 Dataset4 隐式残差	7
3.2 qnorm 与有界化	7
3.3 稳定排序与固定 rank grid	8
4 代码结构与数据处理	8
4.1 目录分层与责任边界	8
4.2 锁定重建调用链	9
4.3 数据解析与索引	9
4.4 文件到算法逻辑的可追踪映射	9
5 可复现性与工件校验	10
5.1 时间边界与样本构造	10
5.2 工件谱系与验收顺序	10
5.3 计算资源与内存控制	10
5.4 确定性序列化	10

6 环境配置与运行	10
6.1 目标平台与依赖	10
6.2 安装与运行准备	11
6.3 框架合规与环境预检	11
6.4 记录结果复验	12
6.5 原始训练与 fresh 推理	12
6.6 统一命令入口与输出检查	12
7 参数、榜单适配与复验结论	13
7.1 关键超参数	13
7.2 A 榜到 B 榜的改动	13
7.3 数据声明与边界	13
7.4 复验结果摘要	14
参考文献	15

1 项目概述

1.1 任务定义与数据边界

赛道一要求对时序交互图中的候选节点进行逐行排序。对第 i 条查询，官方数据提供来源节点、查询时刻和固定宽度候选集合 C_i 。模型为每个候选产生未归一化分数 Z_{ij} ，随后只在当前集合内完成中心化、尺度校准和输出映射：Dataset3 保持概率成员，Dataset4 写入固定 rank grid。两者都保持长度固定、候选列对齐和有限数值。

时间约束是任务的关键。训练阶段按时间建立历史结构，再由历史边构造正负样本和验证查询；排序阶段仅使用官方查询中的来源、时刻和候选结构，以及复验路径所需的模型工件。候选外节点不参与单条查询的分数比较，模型也不为单条查询学习独立参数。

1.2 双数据集建模思路

两套数据集共享“官方输入—学习或统计成员—候选内融合—规范封装”的高层接口。Dataset3 以目标出现频次提供低幅度结构先验，Dataset4 以 MF32 表达来源条件化偏好；二者均在固定候选集合内完成校准，不引入候选外的查询标签。B 榜针对更大的实体规模使用连续索引、排序查找与流式批量。

表 1 模型组件与职责

组件	建模对象	B 榜职责
图排序成员	时序图统计、来源/目标 embedding	学习局部候选偏好
隐式偏好成员	来源—目标点积与目标偏置	个性化候选分数
目标统计	目标频次与近期出现	结构先验
候选集合层	同一查询的集合上下文	候选内校准与排序
融合层	基座分数与模型残差	尺度对齐与幅度控制
序列化层	概率网格与压缩成员	规范提交封装

1.3 方法创新点与设计取舍

第一，保留 A 榜“学习成员 + 候选内结构校正”的范式，把新增计算限制在既有候选集合内部。第二，使用排序索引与流式批处理替代大表的全量中间特征，降低规模增长带来的内存压力。第三，新增残差统一经过 $qnorm$ 与 $tanh$ 有界化，避免统计先验压过主要模型分数。

2 模型与算法

2.1 Dataset3: 目标频次结构残差

Dataset3 不需要来源条件化 embedding。构建器从官方历史边的 dst 列统计每个候选的出现次数，再通过排序词表和 searchsorted 映射到当前查询候选列；未命中项使用零计数，保证映射总有定义。

$$h(c) = \log(1 + n(c)) \quad (1)$$

每行候选先计算 qnorm，再用 tanh 将统计项限制在有界区间，最后以固定残差幅度加入 score-space 基座。该设计使频次只影响候选边界附近的相对次序，同时保留基座的主要排序职责。

$$g_{ij} = \log(1 + n(c_{ij})) \quad (2)$$

$$r_{ij}^{(3)} = \tanh\left(\frac{qnorm(g_{ij})}{2}\right) \quad (3)$$

$$\tilde{z}_{ij}^{(3)} = z_{ij}^{base} + 0.005r_{ij}^{(3)} \quad (4)$$

2.2 Dataset4: 稀疏交互表示与隐式偏好

Dataset4: 训练与推理双路径

同一组候选索引贯穿模型前向、候选内融合与固定序列化



图 1 Dataset4 训练与推理双路径

Dataset4 的主要学习成员为带目标偏置的 32 维隐式矩阵分解。其中 u_s 和 v_c 分别是来源与目标的共享 embedding， b_c 表示目标总体活跃度。B 榜扩展词表并改用排序查找，参数语义和前向计算保持一致。

$$f(s, c) = u_s^T v_c + b_c \quad (5)$$



2.3 负采样与训练目标

训练边按历史时间顺序组织，正边与随机负边形成候选组，AdamW 优化相对排序目标。若每条正边与同组负边记为候选集合 C_i ，并约定第 0 列为正目标，则候选组交叉熵与批量平均损失为：

$$\ell_i = -\log \left(\frac{\exp f(s_i, c_{i,0})}{\sum_{k=0}^{K-1} \exp f(s_i, c_{i,k})} \right) \quad (6)$$

$$L_{MF32} = \frac{1}{B} \sum_{i=1}^B \ell_i \quad (7)$$

$$\theta_{e+1} = \text{AdamW}(\theta_e, \nabla_{\theta} \bar{\mathcal{L}}) \quad (8)$$

训练成员对每条正边抽取 8 个不重复负目标，随机种子由训练入口固定。推理阶段不运行优化器、负采样或超参数搜索，只加载状态并完成批量前向。

2.4 参数规模与流式推理

训练状态包含 source 表、item 表和目标偏置三组参数。若 $|S|$ 、 $|I|$ 分别表示包含保留索引后的表行数，则参数量为：

$$N_{\text{param}} = d(|S| + |I|) + |I| = 50,234,663 \quad (9)$$

加载器将三组数组逐项恢复为 Jittor float32 张量。推理仅 gather 当前批次的 source 与 100 个候选，不构造完整邻域张量。batch_size=4096 用于训练，predict_batch=2048 用于锁定推理。

3 候选内校准、融合与输出

3.1 Dataset4 隐式残差

Dataset4 的确定性残差由 MF32 负责。构建器按批次读取来源和候选目标，调用 Jittor 前向得到 $f(s,c)$ ，再执行候选内 qnorm。最终分数写作：

$$z_{ij} = z_{ij}^{\text{base}} + 0.02 \tanh \left(\frac{\text{qnorm}(f(s_i, c_{ij}))}{2} \right) \quad (10)$$

3.2 qnorm 与有界化

qnorm 在每一行候选内计算均值和尺度，并以 10^{-6} 下限避免零方差除法。tanh(x/2) 将残差限制在 (-1,1)，从而使学习成员只修正候选边界，而不改变候选集合。

$$\mu_i = \frac{1}{K} \sum_{j=1}^K s_{ij} \quad (11)$$

$$\sigma_i = \sqrt{\frac{1}{K} \sum_{j=1}^K (s_{ij} - \mu_i)^2} \quad (12)$$

$$qnorm_i(s_{ij}) = \frac{s_{ij} - \mu_i}{\max(\sigma_i, 10^{-6})} \quad (13)$$

$$r_{ij} = \tanh\left(\frac{qnorm_i(s_{ij})}{2}\right), -1 < r_{ij} < 1 \quad (14)$$

候选内有界残差: $\tanh(qnorm / 2)$

残差只修正基座排序的边界, 不改变候选集合之外的对象

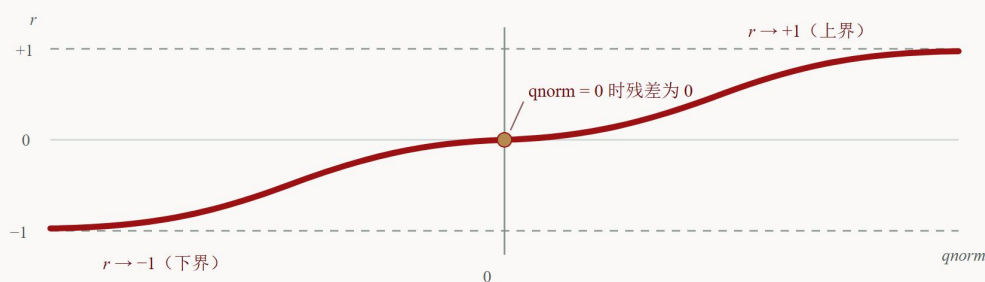


图 2 候选内标准化分数的有界残差

3.3 稳定排序与固定 rank grid

融合后采用稳定降序排序; 候选分数相同时保持原始列顺序。Dataset4 排序结果映射到固定 rank grid:

$$w_{i,o_i(k)} = 1 - \frac{k}{K-1}, k = 0, 1, \dots, K-1, K = 100 \quad (15)$$

4 代码结构与数据处理

4.1 目录分层与责任边界

交付包采用“锁定重建层—原始训练层—工具与审计层”的分层结构。锁定层保存记录结果所需的模型与基座工件; 训练层保留由历史交互构建 Jittor 成员的代码; 工具层负责输入检查、模型状态校验、概率格式和压缩封装。

```
contest1_sader_002/
├── code/main.py                # 统一入口
├── code/build_submission.py    # 构建、融合与序列化
├── code/model.py              # ImplicitMF 定义与加载
├── code/train_model.py        # 历史边、负采样、AdamW
├── code/audit_package.py      # 成员、哈希与结构审计
├── raw_training/main.py       # MF32 训练与 base 生成协调
├── raw_training/build_base.py  # 官方 dst 频次 base
├── requirements.txt           # 固定依赖
├── run_verify.sh              # 记录结果复验入口
└── models/frozen_base.ckpt    # 锁定 score-space 工件
```

4.2 锁定重建调用链

统一入口先验证包内源码、模型状态、基座成员和配置摘要，再读取官方数据并调用结果构建器。构建器依次完成候选读取、基座解析、Dataset4 Jittor 前向、候选内 qnorm、残差融合、稳定排序与规范 ZIP 写出。每一步保留形状、有限性和内容摘要，任一环节不满足记录协议即停止。

4.3 数据解析与索引

输入解析固定列名与 dtype，再以排序词表建立 source、item 连续索引；未命中实体按协议保留特殊索引。候选矩阵按行分块读取，分数数组写出前统一转换为 float32，避免 Python 标量或平台默认类型改变归约顺序。

4.4 文件到算法逻辑的可追踪映射

表 2 核心文件与算法职责

文件	算法职责
code/model.py	ImplicitMF、词表映射、Jittor 前向
code/train_model.py	历史交互、负采样、AdamW、状态写出
code/build_submission.py	基座解析、qnorm、残差融合、排序与 ZIP
code/audit_package.py	成员、形状、摘要与包结构检查
run_train.sh	训练阶段入口
run_inference.sh	记录结果重建入口
run_fresh_inference.sh	新模型推理入口

5 可复现性与工件校验

5.1 时间边界与样本构造

训练器读取官方历史边，建立来源与目标的排序词表，再构造正边与负边。训练边按官方历史输入建立词表，每条正边抽取 8 个不重复负目标，随机种子由入口固定。训练阶段执行一个完整历史历元；推理阶段不运行优化器、负采样或超参数搜索。

5.2 工件谱系与验收顺序

验收链 源码与 MANIFEST → requirements 与 CUDA/Jittor 预检 → 官方 `data_B.zip` SHA → frozen base 与 MF32 checkpoint SHA → 成员行数、宽度与有限性 → CSV 成员摘要 → ZIP CRC → 最终 SHA-256

所有入口均使用非覆盖输出目录；出现缺失、摘要不符或结果不一致时，进程返回失败。模型工件、训练收据、锁定推理收据和最终结果分别记录摘要，从而把训练运行与记录结果复验分开追踪。

5.3 计算资源与内存控制

大规模候选排序采用流式批处理。NumPy 负责排序查找、分组和序列化，Pandas 负责表格分块读取，Jittor 负责 embedding gather、逐维乘加、损失和优化器。每个批次完成前向、qnorm、残差融合和概率写出后释放中间张量，避免构造全量候选分数矩阵。

5.4 确定性序列化

构建器固定成员名 `dataset3.csv`、`dataset4.csv` 及其写出顺序，CSV 使用统一小数格式。ZIP 使用固定成员时间戳和压缩参数，并在关闭归档后重新读取成员计算 SHA-256。数值生成、文件封装和摘要校验因此具有可定位的失败边界。

6 环境配置与运行

6.1 目标平台与依赖

目标平台为 Ubuntu 22.04、NVIDIA RTX 4090、CUDA 12.4 兼容驱动与工具链、Python 3.10、Jittor 1.3.10.0。固定依赖包括 NumPy 1.26.4、Pandas 2.2.3、Numba 0.66.0、nvidia-cudnn-cu12 8.9.7.29、jittor-geometric $\geq$ 0.1.0 与 scikit-learn 1.5.2。

表 3 主要依赖及用途

依赖	用途
Jittor	embedding、逐维乘加、交叉熵、AdamW
NumPy	排序索引、qnorm、概率格式与哈希输入
Pandas	历史交互与查询表分块读取
Numba	大规模索引与数值辅助
nvidia-cudnn-cu12	系统 CUDA 未提供 cuDNN 时的兼容运行库
jittor-geometric	赛道指定的 Jittor 时序图扩展库，按框架合规要求固定安装
scikit-learn	赛道指定的通用机器学习库，供数据校验与度量使用

6.2 安装与运行准备

```
sudo apt-get install -y python3.10 python3.10-venv python3.10-dev build-essential unzip
export CUDA_HOME=/usr/local/cuda-12.4
python3.10 -m venv /data1/sader-repro-py310
source /data1/sader-repro-py310/bin/activate
python -m pip install --upgrade pip
python -m pip install -r requirements.txt
```

安装后检查 `python --version`、`nvcc --version`，并在 Python 中验证 `jittor.__version__`、`jt.has_cuda` 和基本张量算术。模型状态和结果路径由调用参数指定，不依赖某台机器的绝对路径。

特殊依赖说明：JittorGeometric（`jittor-geometric`）已在 `requirements.txt` 中以 `jittor-geometric>=0.1.0` 固定；若 PyPI 无法直接安装，可在安装 Jittor 后从其官方源码仓库获取并执行 `python -m pip install .` 完成本地安装，随后以 `python -c "import jittor_geometric"` 验证。`scikit-learn` 以 `scikit-learn==1.5.2` 固定，随环境一并安装。

6.3 框架合规与环境预检

Jittor 负责 embedding、点积、交叉熵与 AdamW；NumPy/Pandas 负责索引、表格分块和序列化。未引入 PyTorch、TensorFlow 或 Paddle。启动器先执行 `prepare_cuda_runtime.sh`，再执行 `check_environment.py`；若系统 CUDA 缺少 cuDNN，脚本使用 `requirements.txt` 安装的兼容运行库，不修改系统 CUDA。

6.4 记录结果复验

```
bash run_verify.sh /path/to/data_B.zip /data1/sader-byte-exact 0
```

目标 **result.zip · SHA-256** 9a8867eed4bc8a63c203a82ec4e4d5b37c01ebd57894c39c88296334fc13d9ba

`run_verify.sh` 先审计包内代码与资产，再按批次读取官方 `dataset3/4 test.csv`、冻结 `base` 与 `MF32`；构建器写出两个固定成员，并以目标摘要判定结果复验。

6.5 原始训练与 fresh 推理

```
bash run_train.sh /path/to/data_B.zip /path/to/model-output 0
bash run_fresh_inference.sh /path/to/data_B.zip /path/to/model-output /path/to/fresh-output 0
```

`raw_training/main.py` 从官方 `dataset4/train.csv` 建立词表和样本，训练新的 `MF32 checkpoint`；
`raw_training/build_base.py` 从 `dataset3/train.csv` 与 `dataset4/train.csv` 生成 `fresh popularity base`，并写出训练与基座收据。训练目录和结果目录相互独立，不覆盖锁定工件。

6.6 统一命令入口与输出检查

```
python code/main.py verify --data /path/to/data_B.zip --output /path/to/verify-output --gpu 0
python code/main.py infer --data /path/to/data_B.zip --output /path/to/infer-output --gpu 0
python code/main.py train --data /path/to/data_B.zip --output /path/to/model-output --gpu 0
python code/main.py fresh-infer --data /path/to/data_B.zip --models /path/to/model-output --output /path/to/fresh-output --gpu 0
```

表 4 运行接口、输入、输出与验证目标

接口	输入	输出	验证目标
verify	官方数据与保留工件	规范结果与构建收据	记录结果复验
infer	官方数据与保留工件	规范结果与构建收据	直接锁定构建
train	官方历史交互	Jittor 状态与训练收据	新模型训练
fresh-infer	官方数据与新状态	规范结果与推理收据	新模型格式验证

7 参数、榜单适配与复验结论

7.1 关键超参数

模块	参数	含义
Dataset3 popularity	dst 频次; log1p; qnorm; tanh(x/2); residual=0.005	官方训练统计残差
Dataset4 MF32	dim=32; negatives=8; epochs=1; batch=4096; lr=0.001; weight decay=1e-6	隐式偏好学习
MF32 残差	qnorm; tanh(x/2); residual=0.02	来源条件化候选修正
序列化	固定小数位; 稳定排序; 固定成员顺序	规范提交封装

7.2 A 榜到 B 榜的改动

表 5 A/B 算法改动总览

方面	A 榜实现	B 榜实现
高层流程	时序图学习 → 候选内融合 → 规范输出	保持一致
学习成员	32 维来源/目标偏好成员	扩展实体词表, 保持 32 维语义
结构统计	来源支持与候选共现	来源支持 + 目标频次索引
数据组织	候选批量与局部历史	排序词表与流式批量
数值后处理	qnorm、固定残差、稳定排序	保持一致, 重新固定残差幅度
序列化	两个 CSV 成员、固定 ZIP	保持一致

7.3 数据声明与边界

训练、推理、复验与结果封装遵循官方数据接口, 代码读取官方历史交互、查询时刻、来源节点与候选列; 测试集 `ground truth` 不进入代码、模型状态、收据或结果生成流程。交付包不包含官方 `data_B.zip` 或最终 `result.zip`。

原始训练路径从官方数据独立建立新的运行状态, 数值可能受操作系统、CUDA/Jittor 编译、依赖版本、随机初始化和 GPU 归约顺序影响; 该路径的收据用于记录本次运行。记录结果复验由固定资产与固定配置的 `run_verify.sh` 路径承担。

7.4 复验结果摘要

表 6 记录结果与工件摘要

项目	值
线上分数	1.5240999401892983
结果 ZIP SHA-256	9a8867eed4bc8a63c203a82ec4e4d5b37c01ebd57894c39c88296334fc13d9ba
Dataset3 成员 SHA-256	08b2288d63cc265d52ce474cfda2897b1d359b6deb402b73a20e90cc4959a59d
Dataset4 成员 SHA-256	662df72f1df61198ea78787fe5502948a79de918d8c831494df2736b378e5311
MF32 checkpoint SHA-256	98dc703a0851229f38b43f588b709c1b1aef98ab60570a1ca61d8e617eb31f4
Frozen base SHA-256	e46182a6114b0089b9e05d03672b93c28758624ef02b7d97357b1994cddf3d18
A 榜参考包 SHA-256	d159f406a4b6376eb29ebe5b7d54c2481094706dd9f6c67987792cb62966e275

结论 B 榜延续 A 榜的 Jittor 学习成员、候选内 qnorm、固定残差和确定性序列化，并针对实体规模、批量组织和统计索引完成工程适配；原稿记录的线上结果与文件摘要可按同一调用图复验。



参考文献

- [1] YI L, LEI R, MO F, et al. Future link prediction without memory or aggregation[C]//Advances in Neural Information Processing Systems. 2025, 38.
- [2] ROSSI E, CHAMBERLAIN B, FRASCA F, et al. Temporal graph networks for deep learning on dynamic graphs[EB/OL]. arXiv:2006.10637, 2020[2026-08-24]. <https://arxiv.org/abs/2006.10637>.
- [3] HU S M, LIANG D, YANG G Y, et al. Jittor: a novel deep learning framework with meta-operators and unified graph execution[J]. Science China Information Sciences, 2020, 63(12): 222103. DOI:10.1007/s11432-020-3097-4.
- [4] RENDLE S, FREUDENTHALER C, GANTNER Z, et al. BPR: Bayesian personalized ranking from implicit feedback[C]//Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence. Arlington: AUAI Press, 2009: 452-461.
- [5] KOREN Y, BELL R, VOLINSKY C. Matrix factorization techniques for recommender systems[J]. Computer, 2009, 42(8): 30-37. DOI:10.1109/MC.2009.263.
- [6] LOSHCHILOV I, HUTTER F. Decoupled weight decay regularization[C]//International Conference on Learning Representations. 2019.
- [7] XU D, RUAN C, KORPEOGLU E, et al. Inductive representation learning on temporal graphs[C]//International Conference on Learning Representations. 2020.